Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

Отчёт по лабораторной работе №2 по дисциплине «Технологии и инструментальные средства проектирования интеллектуальных систем»

Предметная область «Языки программирования»

Выполнил: Казаченко В.А.
Гр. 321702
Проверил: Гракова

Минск 2026

1 Цель работы

Изучить принципы работы классификатора сообщений, приобрести навыки определения классов сообщений, выделения сущностей и признаков в них, формально специфицировать сценарий диалога.

2 Индивидуальный вариант

Вариант 26 — предметная область «Языки программирования». Фрагмент базы знаний по этой предметной области разработан в лабораторной работе №1 и содержит 23 класса, 17 отношений и 40 экземпляров.

3 Задание

— описать сценарий диалога по предметной области из первой лабораторной работы; сценарий должен состоять минимум из 8 сообщений — 4 вопроса по теме и 4 соответствующих ответа;
— обучить классификатор сообщений Wit.ai на вопрос «Что такое …?» для каждой формализованной сущности и протестировать с помощью вопроса в диалоговом окне Ники;
— выделить классы и сущности сообщений, используемых в описанном сценарии диалога; каждое сообщение должно иметь хотя бы один класс;
— составить отчёт, где указать цель, задание, вариант и показать тестирование сценария диалога;
— уметь отвечать на вопросы и решать задачи, указанные в конце лабораторной работы;
— запушить изменения в свой форк на GitHub.

4 Сценарий диалога

Разработанный сценарий состоит из восьми сообщений: четырёх вопросов пользователя и четырёх ответов системы. Вопросы подобраны так, чтобы задействовать все три реализованных класса сообщений и разные

участки базы знаний: определение понятия, разбиение понятия на виды и факт об авторстве языка.

Таблица 1 — Сценарий диалога пользователя с системой

№	Участник	Сообщение
1	Пользователь	Что такое компилятор?
2	Ника	компилятор - Компилятор — программа, переводящая исходный текст программы с языка программирования в эквивалентный код на языке более низкого уровня, чаще всего в машинный код..
3	Пользователь	Кто создал Python?
4	Ника	Автор языка программирования Python — Гвидо ван Россум.
5	Пользователь	На что декомпозируется язык программирования?
6	Ника	язык программирования разбивается на логический язык программирования, процедурный язык программирования, функциональный язык программирования, объектно-ориентированный язык программирования.
7	Пользователь	Что такое виртуальная машина?
8	Ника	виртуальная машина - Виртуальная машина — программная среда, исполняющая программы, представленные в виде промежуточного (байт-)кода, и изолирующая их от конкретной аппаратной платформы..

Ответы системы строятся не из заранее записанного текста, а из содержимого базы знаний: на первый и четвёртый вопрос подставляется значение отношения nrel_note соответствующего понятия, на третий — элементы отношения nrel_subdividing, на второй — значение отношения nrel_language_author.

На вопрос о декомпозиции система выводит одно из четырёх разбиений понятия, заданных в базе знаний отношением nrel_subdividing.

5 Классы и сущности сообщений

Классификация сообщения по теме означает определение намерения пользователя (класса сообщения) и объекта, о котором идёт речь (сущности сообщения). Класс отвечает на вопрос «что хотят узнать», сущность — «о чём именно спрашивают». Сущность присоединяется к сообщению ролевым отношением rrel_entity.

В сценарии задействованы три класса сообщений. Два из них — штатные классы системы NIKA, третий разработан самостоятельно, поскольку в базе NIKA класса для вопроса об авторе не было.

Таблица 2 — Классы сообщений, используемые в сценарии

Класс сообщения	Вопрос	Правил	Происхождение
`concept_message_about_entity`	Что такое …? / Кто такой …?	58	штатный класс NIKA
`concept_message_about_subdividing`	На что декомпозируется …?	2	штатный класс NIKA
`concept_message_about_author`	Кто создал …?	7	разработан в данной работе

В каждом правиле предусмотрено по две формулировки вопроса, чтобы система распознавала не единственную фразу, а намерение пользователя. Соответствие сообщений сценария классам и сущностям приведено в таблице 3.

Вопрос «Что такое …?» распознаётся для всех 23 классов и 35 экземпляров предметной области; для разработчиков языков он формулируется как «Кто такой …?». Календарные годы в покрытие не включены: в базе знаний они служат значениями отношения nrel_creation_year и самостоятельным предметом вопроса не являются.

Таблица 3 — Классификация сообщений сценария

Сообщение	Класс сообщения	Сущность (rrel_enti
Что такое компилятор?	`concept_message_about_entity`	`concept_compiler`
Кто создал Python?	`concept_message_about_author`	`python_language`
На что декомпозируется язык программирования?	`concept_message_about_subdividing`	`concept_programming_l`
Что такое виртуальная машина?	`concept_message_about_entity`	`concept_virtual_machi`

6 Реализация классификации

6.1 Отказ от сервиса wit.ai

Методические указания предполагают обучение классификатора в облачном сервисе wit.ai. Токен сервиса wit.ai, поставляемый в репозитории системы NIKA, на момент выполнения работы отвергается сервисом, и запросы агента классификации завершаются ошибкой.

```
docker compose logs problem-solver | grep no-auth

{"code":"no-auth","error":"Invalid user: FB log-in user is no longer
supported"}
```

В системе NIKA предусмотрен штатный альтернативный путь — классификация логическими правилами, выполняемая агентом AlternativeMessageTopicClassificationAgent. Этот путь и использован в работе. Изменять конфигурацию для этого не требуется: в программе обработки сообщения агент альтернативной классификации выполняется первым, а обращение к wit.ai происходит только тогда, когда ни одно логическое правило не сработало.

Таким образом, требование задания «обучить классификатор» выполнено, но обучение выражено не размеченными примерами в облачном сервисе, а логическими правилами в самой базе знаний. Вместо экспорта разметки в формате JSON в репозиторий помещены исходные тексты правил в формате .gwf.

6.2 Устройство правила классификации

Каждое правило классификации представляет собой импликацию nrel_implication: посылка описывает образец сообщения, следствие — результат классификации. Посылка содержит дизъюнкцию двух формулировок одного вопроса, следствие относит сообщение к классу и привязывает к нему сущность отношением rrel_entity.

```
ЕСЛИ  текст сообщения = «Что такое компилятор?»
  ИЛИ текст сообщения = «Что значит компилятор?»

ТО    сообщение ∈ concept_message_about_entity
      сообщение → rrel_entity → concept_compiler
```

Правило включено во множество concept_classify_message_rule_priority_1 — именно оттуда агент альтернативной классификации берёт правила для проверки. Поиск по образцу выполняется механизмом template search: система строит из текста сообщения sc-конструкцию и сопоставляет её с посылками правил. Если сопоставление успешно, следствие добавляется в базу знаний.

6.3 Состав разработанных файлов

Все файлы размещены в каталоге фрагмента базы знаний предметной области, разработанного в лабораторной работе №1.

Таблица 4 — Файлы, разработанные в лабораторной работе №2

Путь	Файлов	
`messages/concept_message_about_author.scs`	1	собс сооб в conc
`logic_rules/classification/lr_classify_what_is_*.gwf`	58	прав вопр
`logic_rules/classification/lr_classify_subdividing_of_*.gwf`	2	прав вопр дек
`logic_rules/classification/lr_classify_author_of_*.gwf`	7	прав вопр
`logic_rules/answer/lr_answer_about_author.gwf`	1	прав для сооб
`phrases/concept_phrase_about_author.scs`	1	клас язык
`phrases/examples/concept_phrase_about_author.scs`	1	две обра знач

6.4 Правило ответа для собственного класса

Классификация сообщения и построение ответа — разные механизмы. Для двух штатных классов правила ответа и фразы уже присутствуют в базе знаний NIKA, поэтому первый, третий и четвёртый вопросы сценария получают ответ сразу после добавления правил классификации. Для

собственного класса concept_message_about_author в базе нет ничего, поэтому дополнительно разработаны правило ответа и две фразы.

```
Автор языка программирования ${_language_name} — ${_author_name}.
Язык программирования ${_language_name} создал ${_author_name}.
```

Фраза связана с контуром-образцом отношением nrel_phrase_template. Контур описывает, что нужно найти в базе знаний: сообщение, выделенную в нём сущность-язык, её русский идентификатор, значение отношения nrel_language_author и русский идентификатор этого значения. Найденные значения подставляются в фразу вместо переменных. Формально разработка правил ответа относится к лабораторной работе №3, однако без них третий тип вопроса остаётся без ответа, поэтому они выполнены заранее.

7 Тестирование

Тестирование выполнено на развёрнутой локально системе NIKA: четыре контейнера запущены через docker compose, пользовательский интерфейс доступен по адресу localhost:3033, инструмент просмотра базы знаний sc-web — по адресу localhost:8000.

7.1 Загрузка правил в базу знаний

После копирования файлов в каталог kb/extra решатель перезапущен командой docker compose restart problem-solver; сборка базы знаний завершилась без ошибок. Наличие правил в базе проверено в sc-web.

```
nika $ cd kb/extra/section_subject_domain_of_programming_languages
section_subject_domain_of_programming_languages $ ls
concepts
logic_rules
messages
phrases
relations
section_subject_domain_of_programming_languages.scs
section_subject_domain_of_programming_languages $ ls logic_rules/ messages/ phrases/ logic_rules/answer/
logic_rules/:
answer
classification

logic_rules/answer/:
lr_answer_about_author.gwf

messages/:
concept_message_about_author.scs

phrases/:
concept_phrase_about_author.scs
examples
section_subject_domain_of_programming_languages $ ls logic_rules/classification/ | wc -l
      67
section_subject_domain_of_programming_languages $ ls logic_rules/classification/ | cut -d_ -f3 | sort | uniq -c
   7 author
   2 subdividing
  58 what
```

Рисунок 1 — Структура каталога фрагмента базы знаний

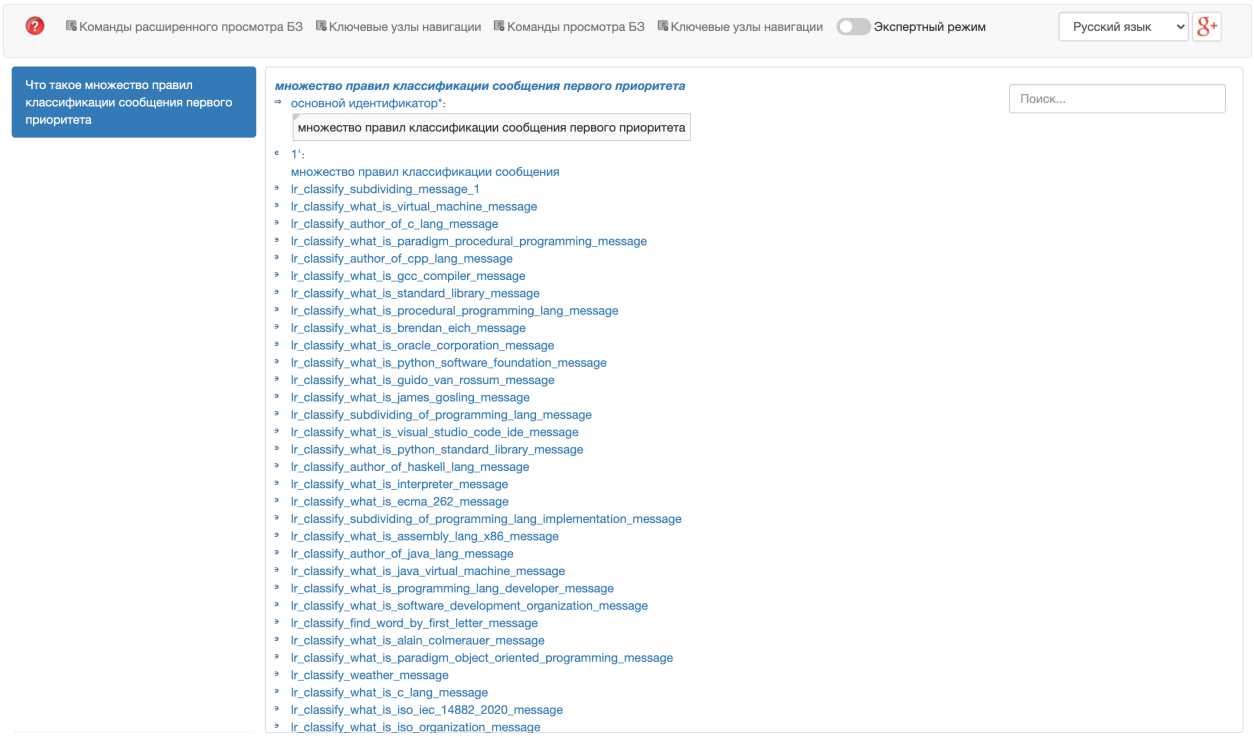


Рисунок 2 — Множество правил классификации concept_classify_message_rule_priority_1

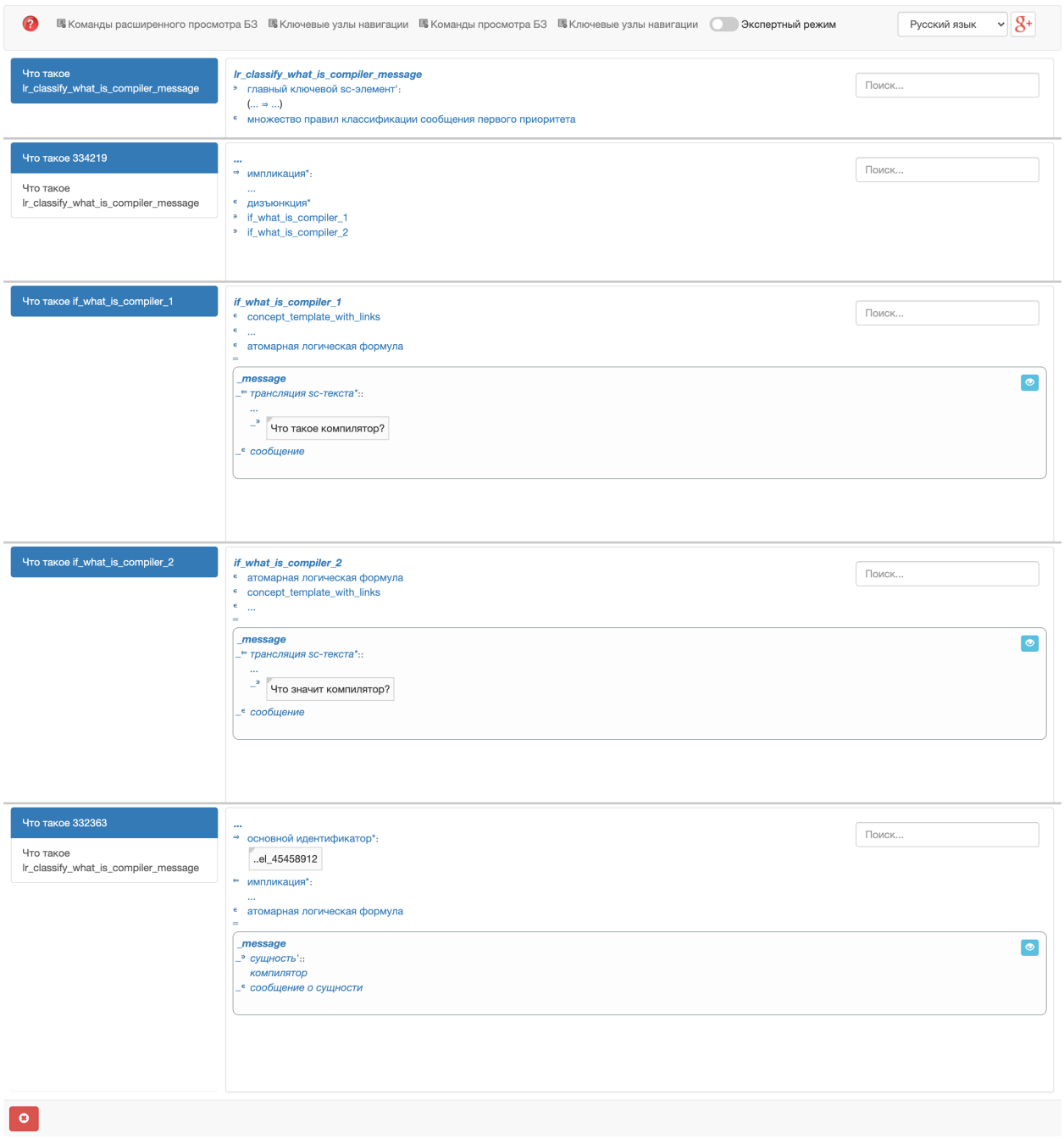


Рисунок 3 — Правило классификации lr_classify_what_is_compiler_message

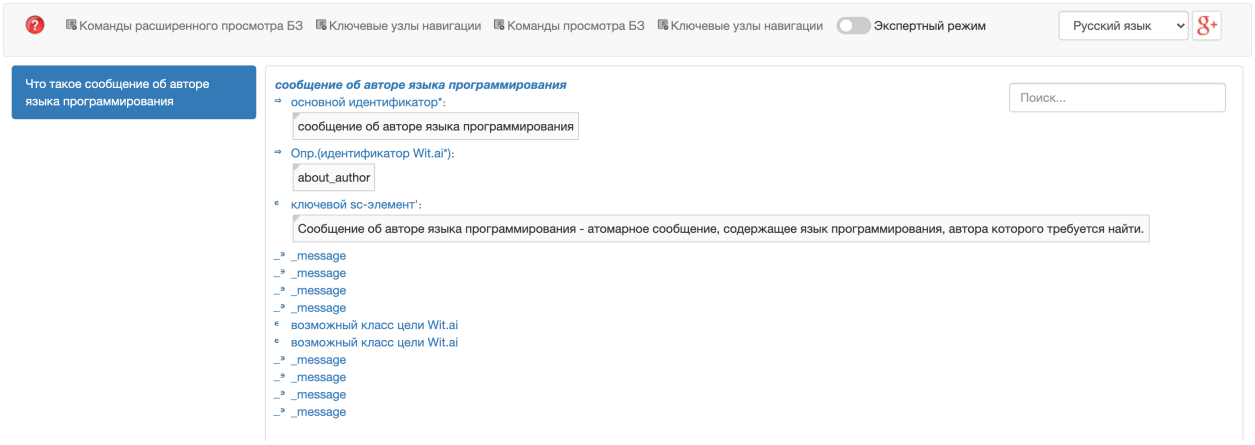


Рисунок 4 — Собственный класс сообщений concept_message_about_author

7.2 Проверка сценария диалога

Сценарий диалога проверен в пользовательском интерфейсе. Все четыре вопроса классифицированы верно, система ответила по существу, используя содержимое базы знаний.

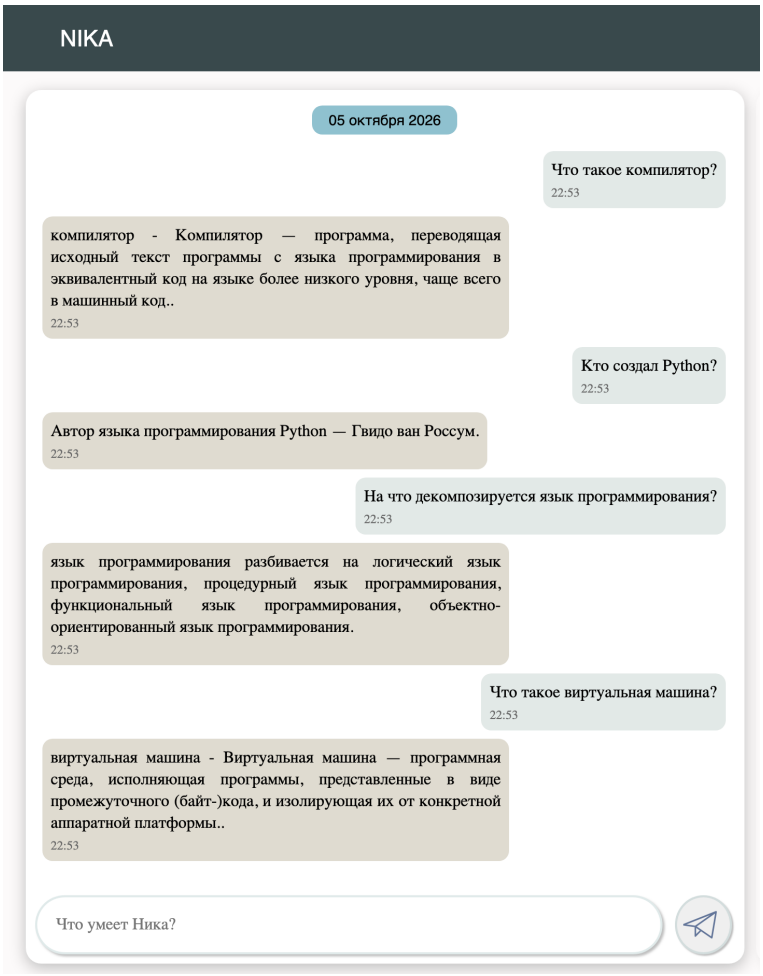


Рисунок 5 — Диалог пользователя с системой по разработанному сценарию

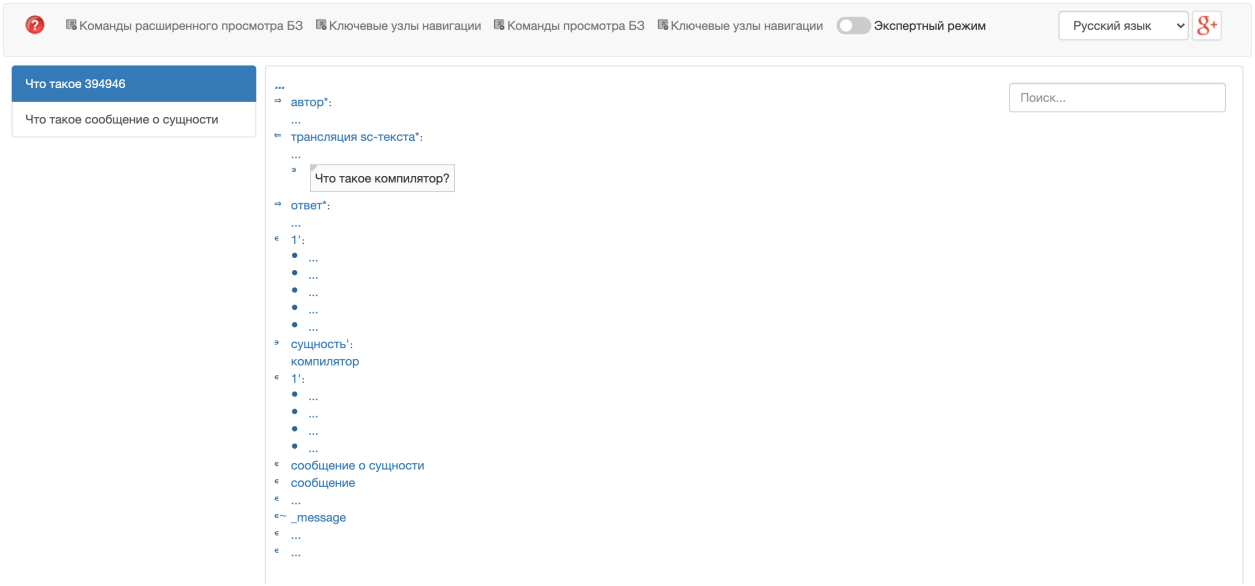


Рисунок 6 — Результат классификации сообщения в базе знаний

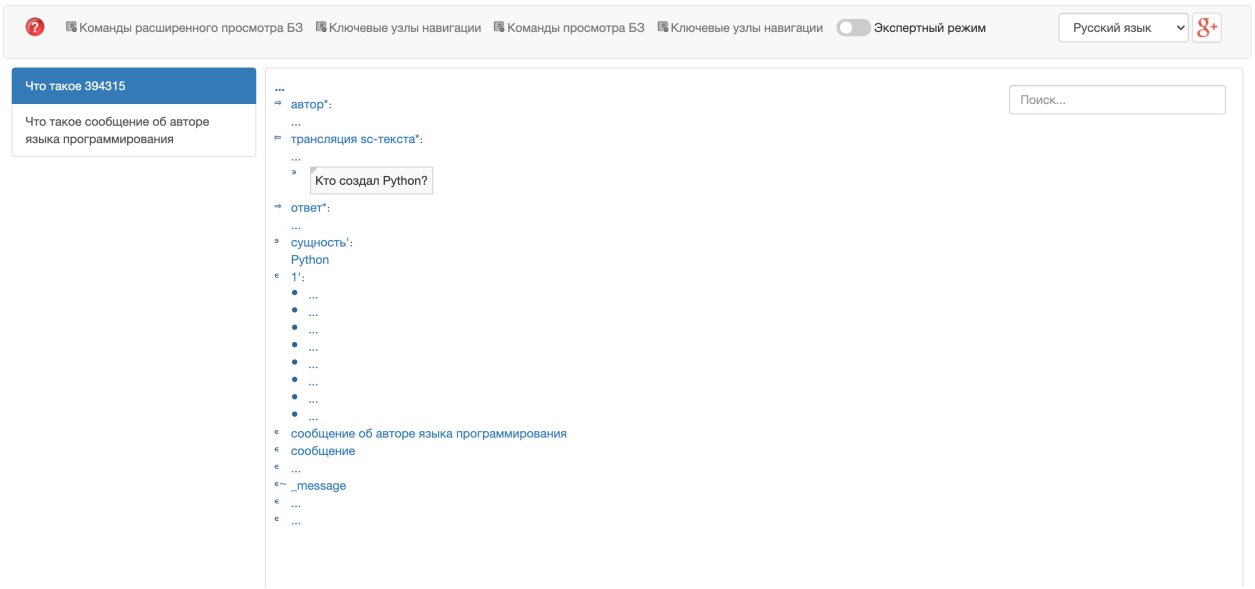


Рисунок 7 — Результат классификации вопроса собственного класса

Появление сообщения в составе класса и наличие у него связи rrel_entity с нужным понятием подтверждают, что классификация выполнена правильно: система определила и намерение пользователя, и объект, о котором задан вопрос.

Помимо сценария, в диалоговом окне проверены все 134 формулировки вопросов из 67 разработанных правил классификации: на каждую система дала ответ по существу. Вопрос, для которого правила нет, например «Кто

создал язык ассемблера x86?», получает одну из штатных фраз непонимания, например «Сформулируйте, пожалуйста, по-другому.».

8 Вывод

В ходе выполнения лабораторной работы разработан сценарий диалога из восьми сообщений по предметной области «Языки программирования», выделены три класса сообщений и соответствующие им сущности, реализована классификация сообщений и проведено её тестирование. Разработано 67 правил классификации, собственный класс сообщений, правило построения ответа и две фразы ответа.

Основная сложность работы состояла в том, что штатный токен сервиса wit.ai не принимается сервисом. Решение найдено в самой системе NIKA: вместо облачного классификатора использован штатный механизм альтернативной классификации логическими правилами. Такой подход оказался даже ближе к идеологии ostis-технологии: знание о том, как распознать сообщение, хранится не во внешнем сервисе, а в самой базе знаний, в том же виде, что и остальные знания системы, и может быть просмотрено и отредактировано теми же средствами.

Работа показала, что понимание сообщения в ostis-системе разделено на два независимых шага: классификация определяет намерение и сущность, а построение ответа выполняется отдельными правилами и фразами. Пока для класса сообщений нет правила ответа, система распознаёт вопрос, но ответить на него не может — именно это и наблюдалось для собственного класса до разработки правила ответа.